

Nome: Luciano Magri

Resolução do problema dos missionários e canibais utilizando o método A*.

Código Python gerado com IA.

```
from heapq import heappush, heappop

# Estado: (m_left, c_left, boat_side)
# boat_side = 0 -> barco na esquerda, 1 -> barco na direita
# No início: 3 missionários, 3 canibais à esquerda, barco à esquerda.
INITIAL_STATE = (3, 3, 0)
GOAL_STATE = (0, 0, 1)
TOTAL_M = 3
TOTAL_C = 3

def estado_valido(state):
    """
    Verifica se um estado é válido:
    - Ninguém fora do intervalo [0, TOTAL]
    - Em qualquer margem, se houver missionários, eles não podem ser
    minoria.
    """
    m_left, c_left, boat = state
    m_right = TOTAL_M - m_left
    c_right = TOTAL_C - c_left

    # Limites
    if not (0 <= m_left <= TOTAL_M and 0 <= c_left <= TOTAL_C):
        return False
    if not (0 <= m_right <= TOTAL_M and 0 <= c_right <= TOTAL_C):
        return False

    # Regra missionários x canibais
    if m_left > 0 and c_left > m_left:
        return False
    if m_right > 0 and c_right > m_right:
        return False

    return True

def sucessores(state):
```

```

"""
Gera todos os estados sucessores válidos de 'state'
com movimentos possíveis do barco (1 ou 2 pessoas).
"""

m_left, c_left, boat = state
moves = [
    (1, 0), # 1 missionário
    (2, 0), # 2 missionários
    (0, 1), # 1 canibal
    (0, 2), # 2 canibais
    (1, 1), # 1 missionário + 1 canibal
]

sucessores = []

for m_move, c_move in moves:
    if boat == 0:
        # Barco vai da esquerda para a direita
        new_state = (m_left - m_move, c_left - c_move, 1)
    else:
        # Barco vai da direita para a esquerda
        new_state = (m_left + m_move, c_left + c_move, 0)

    if estado_valido(new_state):
        sucessores.append(new_state)

return sucessores

def heuristica(state):
    """
    Heurística admissível simples:
    quantidade de pessoas restantes na margem esquerda / 2,
    pois o barco leva no máximo 2 por vez.
    """
    m_left, c_left, boat = state
    pessoas_restantes = m_left + c_left
    return pessoas_restantes / 2.0

def reconstrui_caminho(came_from, current):
    caminho = [current]
    while current in came_from:

```

```

        current = came_from[current]
        caminho.append(current)
    caminho.reverse()
    return caminho

def a_star(initial_state, goal_state):
    """
    Implementação do A*:
    - priority queue com (f, g, state)
    - g = custo acumulado (número de travessias)
    - f = g + h
    """
    open_set = []
    heappush(open_set, (heuristica(initial_state), 0, initial_state)) #
(f, g, (m_left, c_left, boat_side))

    came_from = {} # para reconstruir o caminho
    g_score = {initial_state: 0}

    closed_set = set()

    while open_set:
        f, g, current = heappop(open_set)

        if current == goal_state:
            return reconstrui_caminho(came_from, current)

        if current in closed_set:
            continue
        closed_set.add(current)

        for neighbor in sucessores(current):
            tentative_g = g_score[current] + 1 # cada movimento custa 1

            if neighbor in g_score and tentative_g >= g_score[neighbor]:
                continue

            came_from[neighbor] = current
            g_score[neighbor] = tentative_g
            f_neighbor = tentative_g + heuristica(neighbor)
            heappush(open_set, (f_neighbor, tentative_g, neighbor))

```

```

    return None

def imprime_estado(state):
    m_left, c_left, boat = state
    m_right = TOTAL_M - m_left
    c_right = TOTAL_C - c_left
    barco_esq = "B" if boat == 0 else " "
    barco_dir = "B" if boat == 1 else " "
    return f"Esq: M={m_left}, C={c_left} {barco_esq} | Dir:
M={m_right}, C={c_right} {barco_dir}"

if __name__ == "__main__":
    caminho = a_star(INITIAL_STATE, GOAL_STATE)

    if caminho is None:
        print("Nenhuma solução encontrada.")
    else:
        print("Solução encontrada em", len(caminho) - 1,
"movimentos:\n")
        for i, s in enumerate(caminho):
            print(f"Passo {i}: {imprime_estado(s)}")

```